

Sentinel AI Technical Contract and Integration Blueprint

Team: BYTE STORM

Hackathon: Smart Horizon 2026 International 48 Hour Hackathon

Phase: Pre Event Preparation

1. Purpose of This Document

This document defines the technical contracts that the SentinelAI team must agree on before the hackathon begins.

The objective is to ensure that all five team members build against the same interfaces during the 48-hour event.

The team must finalize:

1. Shared transaction JSON schema
2. API request and response contracts
3. Database schema
4. ML input and output contract
5. Risk Engine contract
6. Repository structure
7. Docker Compose infrastructure skeleton
8. Integration boundaries between team members

2. Team Technical Ownership

Member	Primary Ownership
Thanusree	Backend, FastAPI, PostgreSQL, API contracts, orchestration, Docker, integration, rules and compliance
Alisha	Frontend, authentication screens, transaction simulator UI, transaction detail
Chandana	Frontend, dashboard, charts, monitoring and risk visualization
Devika	Data Science, Isolation Forest, feature engineering, model evaluation
Krrish	AI/ML decision layer, risk formula, rule engine, velocity, receiver risk, reason codes, synthetic fraud scenarios

Integration Principle

Ownership means responsibility for a component, not a separate repository.

The team will use:

ONE GitHub Repository

```
|
+— backend
+— frontend
+— ml
+— simulator
+— docs
```

3. Shared Transaction JSON Contract

Every transaction entering SentinelAI should follow the same structure.

```
{
  "transaction_id": "TXN_000001",
  "user_id": "USR_001",
  "amount": 2500.00,
  "currency": "INR",

  "receiver_id": "REC_045",
  "receiver_type": "merchant",

  "timestamp": "2026-09-03T10:15:30+05:30",

  "device_id": "DEV_019",
  "device_type": "android",

  "location": {
    "latitude": 12.9716,
    "longitude": 77.5946
  },
  "ip_address": "192.168.1.10",
  "user_context": {
    "account_age_days": 420,
    "previous_transaction_count": 157,
    "usual_transaction_range": {
      "min": 100.0,
      "max": 5000.0
    }
  }
}
```

4. Transaction Field Definitions

Field	Type	Required	Purpose
transaction_id	String	Yes	Unique transaction identifier
user_id	String	Yes	Identifies the sender
amount	Decimal	Yes	Transaction amount
currency	String	Yes	Currency code
receiver_id	String	Yes	Receiver identifier
receiver_type	Enum/String	Yes	Merchant, user, bank account, etc.
timestamp	DateTime	Yes	Transaction time
device_id	String	Yes	Device associated with transaction
device_type	Enum/String	Yes	Android, iOS, web, etc.
location.latitude	Decimal	No	Approximate transaction location
location.longitude	Decimal	No	Approximate transaction location
ip_address	String	No	Source IP
account_age_days	Integer	Yes	Age of user's account
previous_transaction_count	Integer	Yes	Historical transaction count
usual_transaction_range	Object	Yes	User's historical transaction range

Naming Rule

The team must use the same field names everywhere.

For example, if the official field is:

receiver_id

then every component must use:

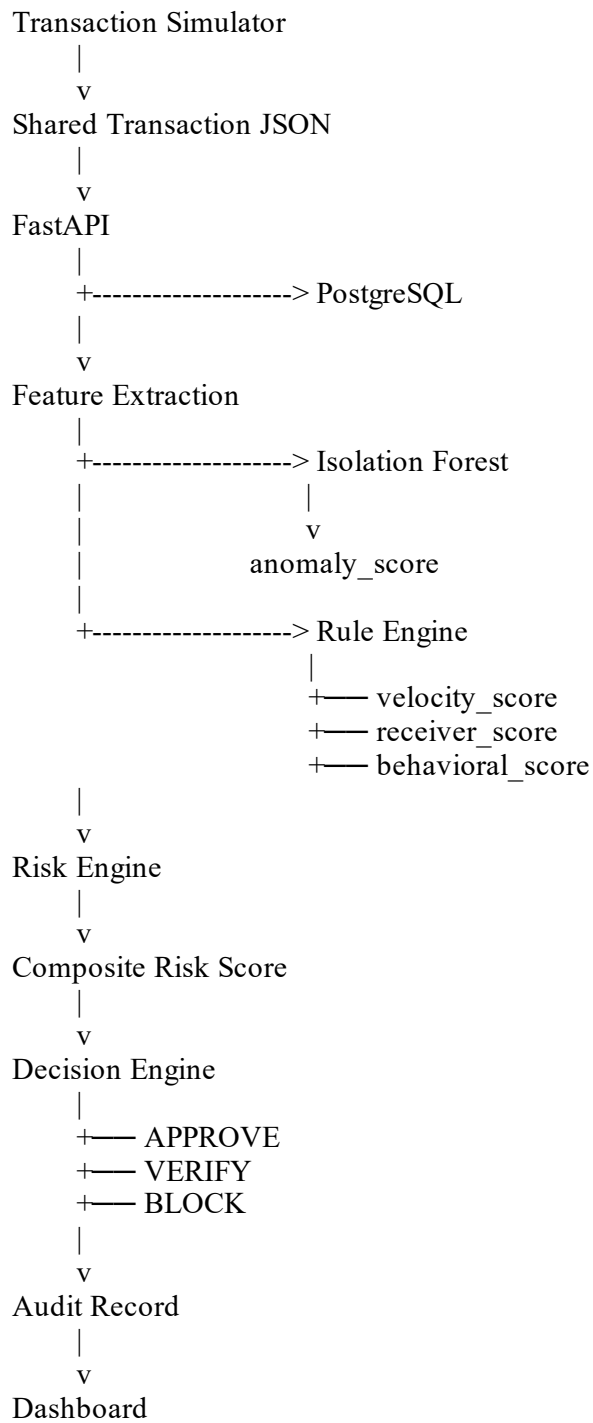
receiver_id

Not:

receiverId
recipient_id
recipientId
payee
merchant_id

One concept gets one name.

5. End to End Transaction Flow



6. ML Contract

Isolation Forest component must consume the shared transaction and feature structure.

The model must eventually return a normalized anomaly score.

Required Output

```
{  
  "anomaly_score": 82.5,  
  "model_version": "iforest_v1",  
  "model_status": "success"  
}
```

Contract

anomaly_score = 0 to 100

The Risk Engine must never have to interpret raw Isolation Forest values.

For example, the downstream system should not receive:

```
-0.42  
0.83  
"highly suspicious"
```

It should receive:

```
82.5
```

The conversion and normalization belong to the ML component.

7. Rule Engine Contract

Rule engine will independently evaluate transaction level risk signals.

Expected output:

```
{  
  "velocity_score": 75.0,  
  "receiver_score": 80.0,  
  "behavioral_score": 60.0,  
  
  "rules_triggered": [  
    "HIGH_TRANSACTION_VELOCITY",  
    "NEW_RECEIVER",  
    "UNUSUAL_AMOUNT"  
  ],  
  
  "reason_codes": [  
    "Multiple transactions detected within a short time window",  
    "Receiver has limited transaction history",  
    "Amount differs significantly from user's normal behavior"  
  ]  
}
```

```
]
}
```

Required normalized signals

velocity_score = 0 to 100
receiver_score = 0 to 100
behavioral_score = 0 to 100

All signals must be normalized before they reach the Risk Engine.

8. Risk Engine Contract

The Risk Engine receives:

- anomaly_score
- velocity_score
- receiver_score
- behavioral_score
- reason_codes

It combines the signals using the finalized weighting formula.

The initial proposed weighting is:

- Anomaly 40%
- Velocity 25%
- Receiver 20%
- Behavior 15%

The final weighting must be agreed upon by the team during the hackathon.

Expected Output

```
{
  "transaction_id": "TXN_000001",

  "risk": {
    "anomaly_score": 82.5,
    "velocity_score": 75.0,
    "receiver_score": 80.0,
    "behavioral_score": 60.0,
    "composite_score": 77.8
  },

  "decision": "BLOCK",
```

```

"reason_codes": [
  "HIGH_ANOMALY",
  "HIGH_TRANSACTION_VELOCITY"
],

"model_version": "iforest_v1",

"evaluated_at": "2026-09-03T10:15:31+05:30"
}

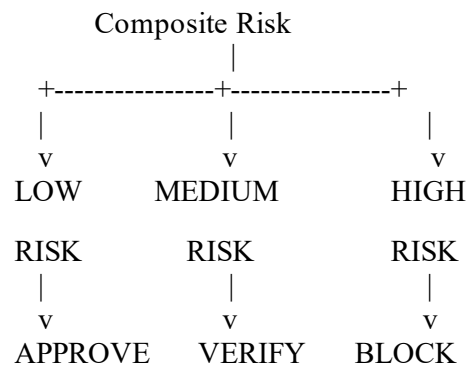
```

9. Decision Engine

The final composite score must map to a decision.

The exact thresholds should be finalized and tested during the hackathon.

Conceptually:



The thresholds must not be quietly changed to force the demonstration of transactions into the desired outcomes.

After Hour 36, any threshold modification must trigger a complete regression and demo test run.

10. Database Schema

The initial database should contain five primary tables:

```

users
transactions
risk_assessments
rule_events
audit_logs

```

10.1 Users

users

id	UUID PK
user_id	VARCHAR UNIQUE
account_age_days	INTEGER
created_at	TIMESTAMP

10.2 Transactions

transactions

id	UUID PK
transaction_id	VARCHAR UNIQUE
user_id	VARCHAR FK
amount	DECIMAL
currency	VARCHAR
receiver_id	VARCHAR
receiver_type	VARCHAR
timestamp	TIMESTAMP
device_id	VARCHAR
device_type	VARCHAR
latitude	DECIMAL NULL
longitude	DECIMAL NULL
ip_address	VARCHAR NULL
created_at	TIMESTAMP

10.3 Risk Assessments

risk_assessments

id	UUID PK
transaction_id	VARCHAR FK
anomaly_score	DECIMAL
velocity_score	DECIMAL
receiver_score	DECIMAL
behavioral_score	DECIMAL
composite_score	DECIMAL
decision	VARCHAR
model_version	VARCHAR
evaluated_at	TIMESTAMP

Purpose:

Stores the complete risk calculation for each evaluated transaction.

10.4 Rule Events

rule_events

id	UUID PK
transaction_id	VARCHAR FK
rule_code	VARCHAR
rule_name	VARCHAR
score	DECIMAL
reason	TEXT
created_at	TIMESTAMP

Purpose:

Stores individual rules triggered during evaluation.

One transaction can trigger multiple rule events.

10.5 Audit Logs

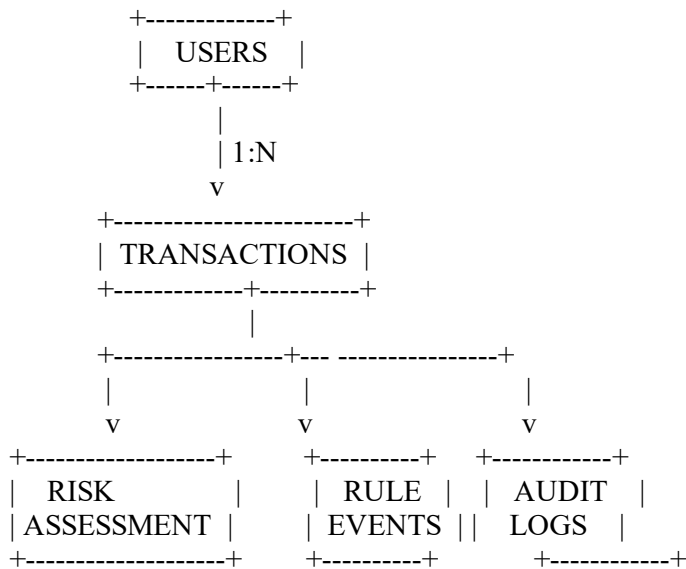
audit_logs

id	UUID PK
transaction_id	VARCHAR FK
event_type	VARCHAR
decision	VARCHAR
risk_score	DECIMAL
details	JSONB
created_at	TIMESTAMP

Purpose:

Provides an audit trail of important system events and decisions.

11. Database Relationship



Relationship principle

One transaction can have:

1 Risk Assessment

Many Rule Events

Many Audit Events

12. API Contract

The initial API contract should contain the following endpoints:

POST /transactions/evaluate

GET /transactions

GET /transactions/{transaction_id}

GET /dashboard/summary

GET /dashboard/risk-distribution

GET /health

12.1 Evaluate Transaction

Endpoint

POST /transactions/evaluate

Input

Shared Transaction JSON.

Processing

Transaction

|

v

Feature Extraction

|

+—— Isolation Forest

|

+—— Rule Engine

|

v

Risk Engine

|

v

Decision Engine

|

v

Database

|

v

Response

Response

```
{
  "transaction_id": "TXN_000001",
  "composite_score": 77.8,
  "decision": "BLOCK",

  "risk_breakdown": {
    "anomaly": 82.5,
    "velocity": 75.0,
    "receiver": 80.0,
    "behavioral": 60.0
  },

  "reason_codes": [
    "HIGH_ANOMALY",
    "HIGH_TRANSACTION_VELOCITY"
  ]
}
```

13. Repository Structure

The project will use a monorepo.

```
sentinel-ai/
|
|-- backend/
|   |-- app/
|   |   |-- api/
|   |   |-- models/
|   |   |-- schemas/
|   |   |-- services/
|   |   |-- risk/
|   |   |-- rules/
|   |   |-- main.py
|   |
|   |-- tests/
|   |-- requirements.txt
|   |-- Dockerfile
|
|-- frontend/
|   |-- components/
|   |-- pages/
|   |-- services/
|   |-- Dockerfile
|
|-- ml/
|   |-- notebooks/
|   |-- models/
|   |-- features/
|   |-- requirements.txt
|
|-- simulator/
|   |-- README.md
|
|-- docs/
|   |-- architecture.md
|   |-- api-contract.md
|   |-- database-schema.md
|   |-- ml-contract.md
|   |-- risk-engine-contract.md
|   |-- demo-scenarios.md
|
|-- docker-compose.yml
|-- .gitignore
|-- README.md
|-- .env.example
```

Ownership

backend/ → Thanusree

frontend/ → Alsiba + Chandana

ml/ → Devika

risk/rules → Krish

14. Docker Compose Skeleton

The Phase 0 Docker configuration should only establish the infrastructure.

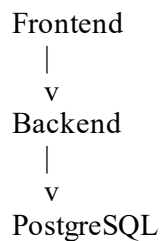
services:

```
backend:
  build: ./backend
  ports:
    - "8000:8000"
  depends_on:
    - postgres
```

```
frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  depends_on:
    - backend
```

```
postgres:
  image: postgres:16
  environment:
    POSTGRES_DB: sentinelai
    POSTGRES_USER: sentinel
    POSTGRES_PASSWORD: sentinel
  ports:
    - "5432:5432"
```

The purpose of this skeleton is to establish that the eventual system can run as:

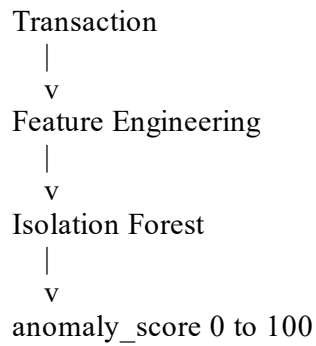


No SentinelAI business logic should be added during Phase 0.

15. Component Boundaries

Devika

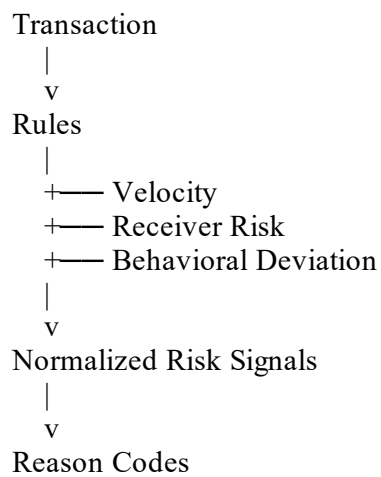
Devika owns:



Devika does not need to know how the dashboard is implemented.

Krrish

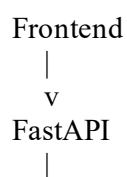
Krrish owns:

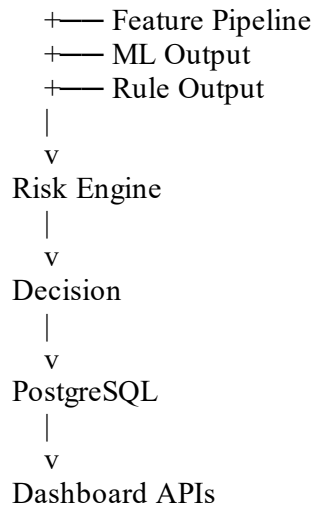


Krrish does not need to directly manage PostgreSQL.

Thanusree

Thanusree owns:





Thanusree is the integration boundary between components.

16. Critical Integration Rule

All risk signals entering the Risk Engine must be normalized to:

0 to 100

This includes:

anomaly_score
velocity_score
receiver_score
behavioral_score

The Risk Engine must not contain component specific conversion logic.

Each upstream component is responsible for delivering a clean, normalized score.

17. Shared Test Transactions

The team will use three standard transactions throughout development.

Transaction A: Normal

Expected:

APPROVE

Characteristics:

Normal amount

Known receiver

Known device

Normal location

Normal transaction frequency

Normal user behavior

Transaction B: Suspicious

Expected:

VERIFY

Characteristics:

Unusual amount

New receiver

Some behavioral deviation

Moderate risk signals

Transaction C: High Risk

Expected:

BLOCK

Characteristics:

Extreme anomaly

High transaction velocity

Suspicious receiver

Strong behavioral deviation

All three transactions must be rerun whenever a major component changes.

18. Cross Account Fraud Scenario

The original coordinated micro transaction scenario requires special treatment.

A per user Isolation Forest cannot automatically detect a pattern where thousands of different accounts each perform an individually normal ₹20 transaction.

Therefore, the team must choose one of two approaches during the hackathon:

Option A: Implement a Cross Account Rule

Krrish adds a rule such as:

Many new accounts
+
Same receiver
+
Short time window
=
High receiver / velocity risk

This becomes an independent rule-based detection capability.

Option B: Treat It as a Limitation

If the rule is not implemented, the team must not claim that the current Isolation Forest architecture detects this scenario.

It should instead be presented as:

Current limitation:

Per user behavioral models do not inherently detect coordinated cross account attacks.

Future direction:

Graph based and cross account behavioral analysis.

The pitch must accurately reflect what the system actually does.

19. Phase 0 Final Documentation Package

Before September 3, the docs/ directory should contain:

```
docs/  
|  
+-- architecture.md  
+-- api-contract.md  
+-- database-schema.md  
+-- ml-contract.md  
+-- risk-engine-contract.md  
+-- demo-scenarios.md
```

These documents define the interfaces the team will implement during the hackathon.

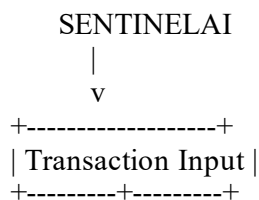
20. Phase 0 Definition of Done

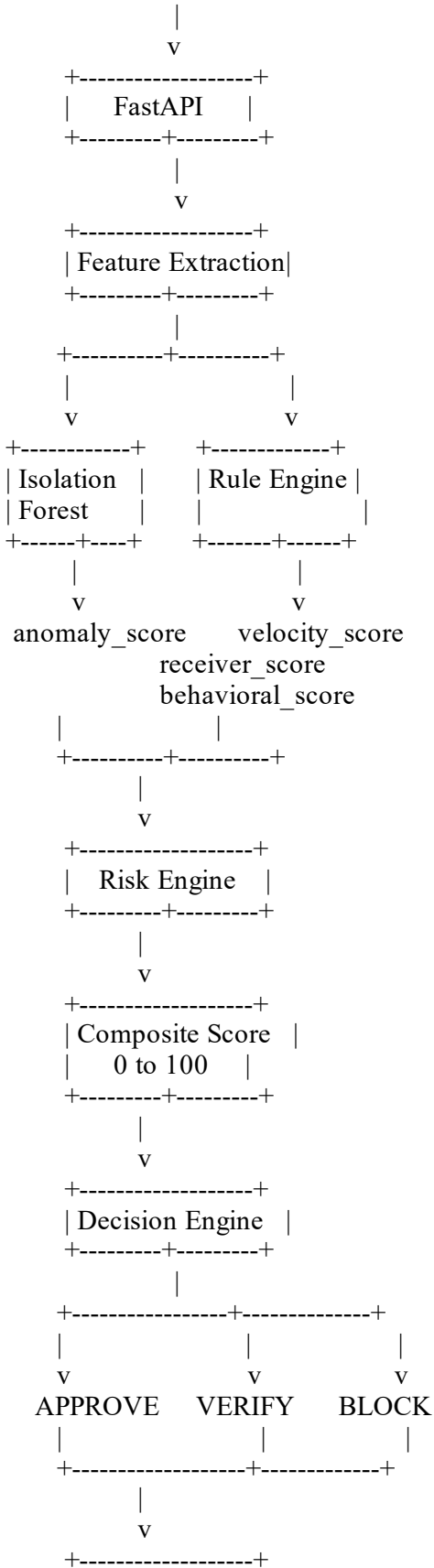
Phase 0 is complete when all of the following are agreed upon:

- [] Shared transaction schema finalized
- [] Field names finalized
- [] Field types finalized
- [] API request/response contracts finalized
- [] Database schema finalized
- [] Database relationships documented
- [] ML input/output contract finalized
- [] Rule Engine output contract finalized
- [] Risk Engine contract finalized
- [] Repository structure created
- [] Docker Compose skeleton created
- [] Ownership boundaries confirmed
- [] Standard test transactions defined
- [] Cross account fraud scenario decision made
- [] All five members understand the integration flow
- [] No SentinelAI application logic has been built before the hackathon

21. Final Integration Blueprint

The complete SentinelAI architecture should ultimately follow this model:





| Audit Logging |

+-----+-----+

|

v

+-----+-----+

| PostgreSQL |

+-----+-----+

|

v

+-----+-----+

| Monitoring
| Dashboard |

+-----+-----+